АВРОРА
СВОЯ СИСТЕМА

Интенсив Flutter разработки под ОС Аврора

День 1 - Основы Flutter

Овчинников Сергей
Инженер-разработчик ОМП

Содержание мастер класса

- Flutter
- Устройства Flutter для ОС Аврора
- Инструменты разработки Flutter под ОС Аврора
- Особенности разработки под ОС Аврора

Flutter

Flutter

Что такое Flutter

1. Flutter — это UI SDK от Google.
2. Создание нативных приложений для 6 платформ из одной базы кода.
3. Единый дизайн-код для Mobile, Web и Desktop.
4. Язык программирования Dart.
5. Открытый исходный код (Open Source).
6. Рекордные темпы роста популярности в индустрии и поддержка сообществом



Flutter

Flutter

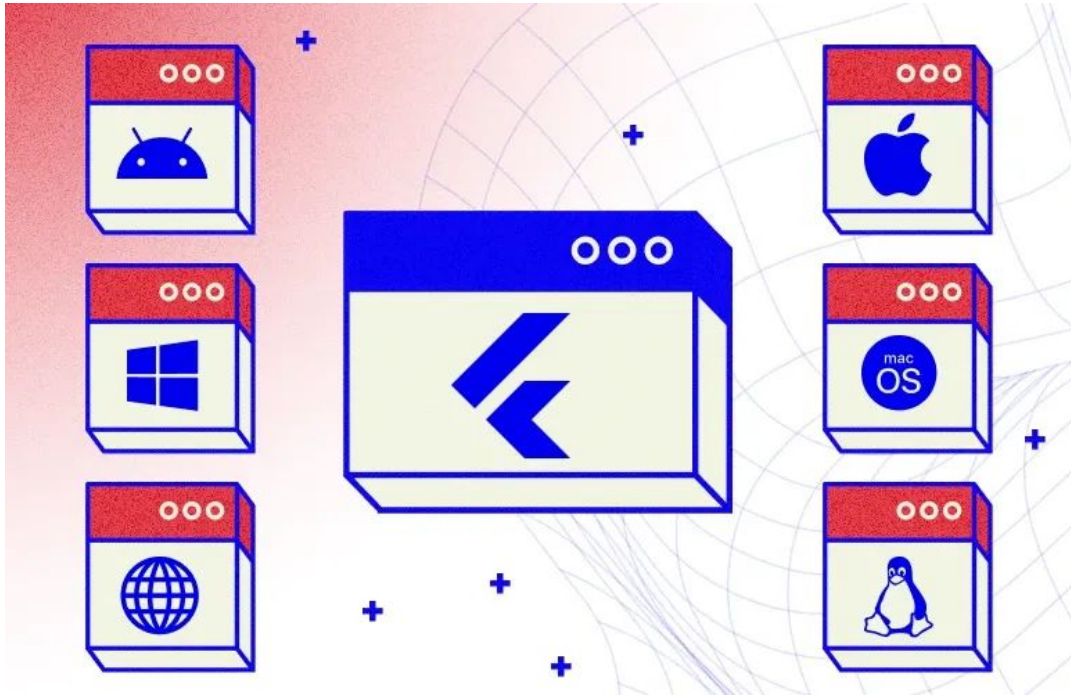
Проблемы и решения

Проблемы:

- Высокая стоимость поддержки двух нативных команд
- Рассинхрон функций: фича вышла на iOS, но задерживается на Android
- Проблема фрагментации устройств и экранов
- Медленный цикл разработки
- Сложность поддержки единого визуального стиля

Решение:

- Единый движок для отрисовки
- Снижение затрат на разработку



Flutter

Язык Dart

1. Объектно-ориентированный язык с C-подобным синтаксисом.
2. JIT-компиляция: быстрая разработка (Hot Reload).
3. AOT-компиляция: высокая производительность в релизе.
4. Strong Typing: меньше ошибок за счет строгой типизации.
5. Sound Null Safety: исключение ошибок "Null Pointer Exception".
6. Встроенный сборщик мусора (Garbage Collector), оптимизированный для UI.
7. Богатая стандартная библиотека для работы с коллекциями и асинхронностью



Flutter

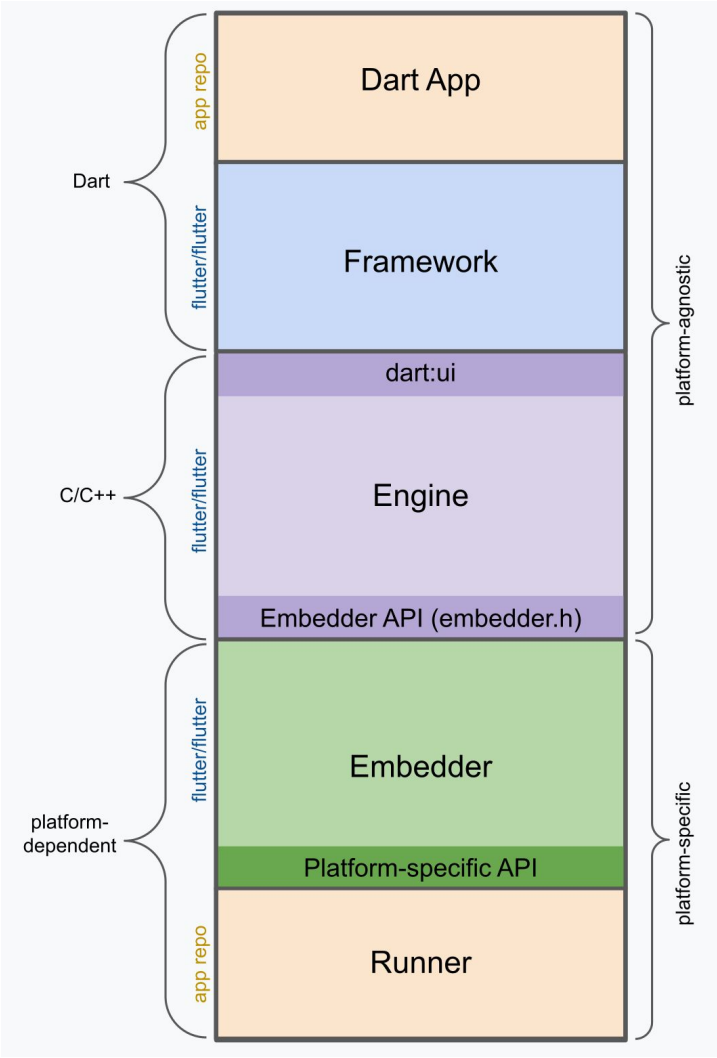
Архитектура Flutter

Архитектурные слои:

1. **Framework (Dart):** управление жестами, анимациями и верстkой.
2. **Engine (C++):** графический движок Impeller/Skia.
3. **Embedder:** слой адаптации под конкретную ОС (Windows, macOS, Linux).

Полезное:

1. **Platform Channels:** способ общения с Bluetooth, GPS и камерой.
2. **Skia/Impeller:** отрисовка каждого пикселя вручную (как в играх).
3. **Text Rendering:** собственный движок обработки шрифтов.

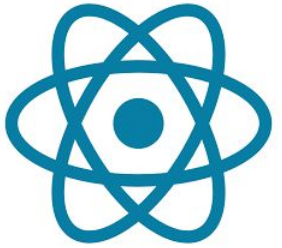


Flutter

Сравнение технологий

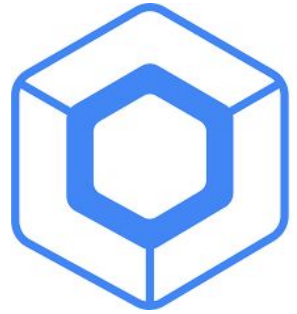
1. **WebView:** Медленно, выглядит как сайт в обертке.
2. **React Native:** Зависимость от системных компонентов и JS-моста.
3. **Flutter:** Полная независимость от OEM-виджетов.
4. **KMP (Kotlin Multiplatform):** Общая бизнес-логика при 100% нативном UI.
5. **CMP (Compose Multiplatform):** Декларативный UI от JetBrains, прямой конкурент Flutter.
6. **Qt**: Высокая производительность на C++, но высокий порог входа.







Kotlin Multiplatform

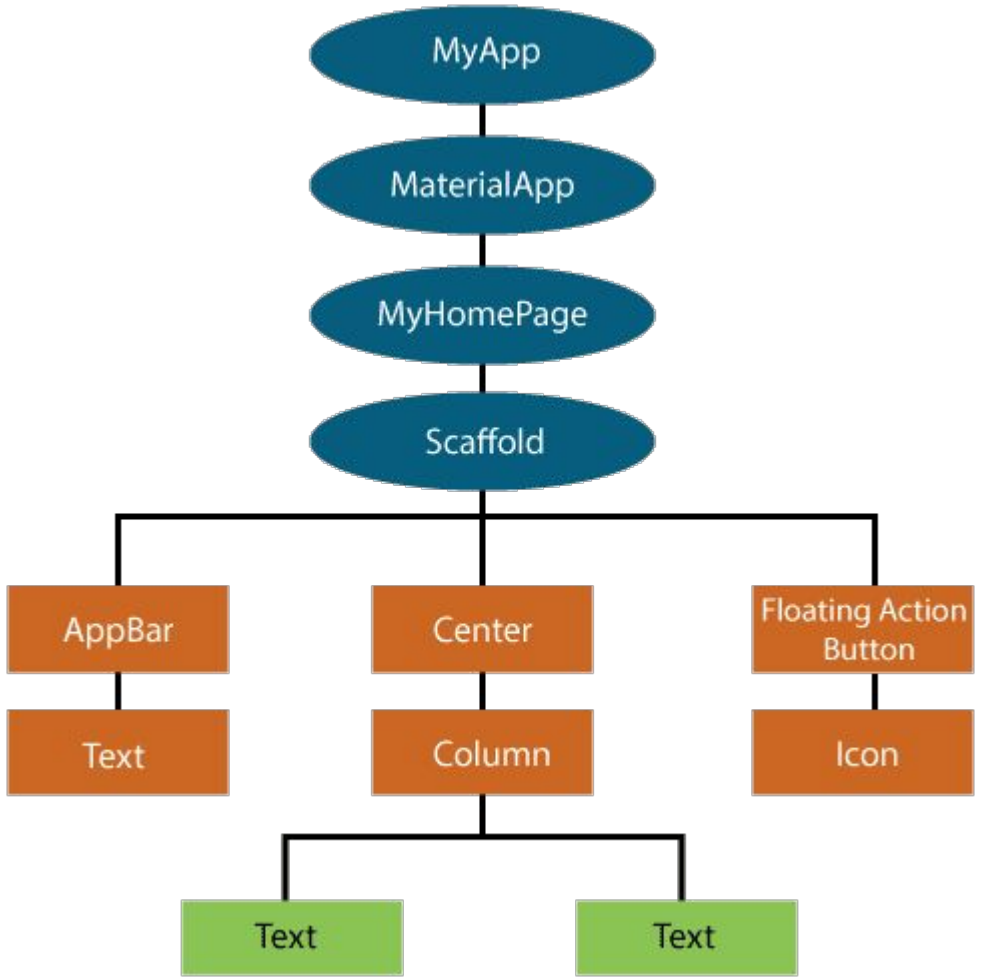




Flutter

Всё виджеты

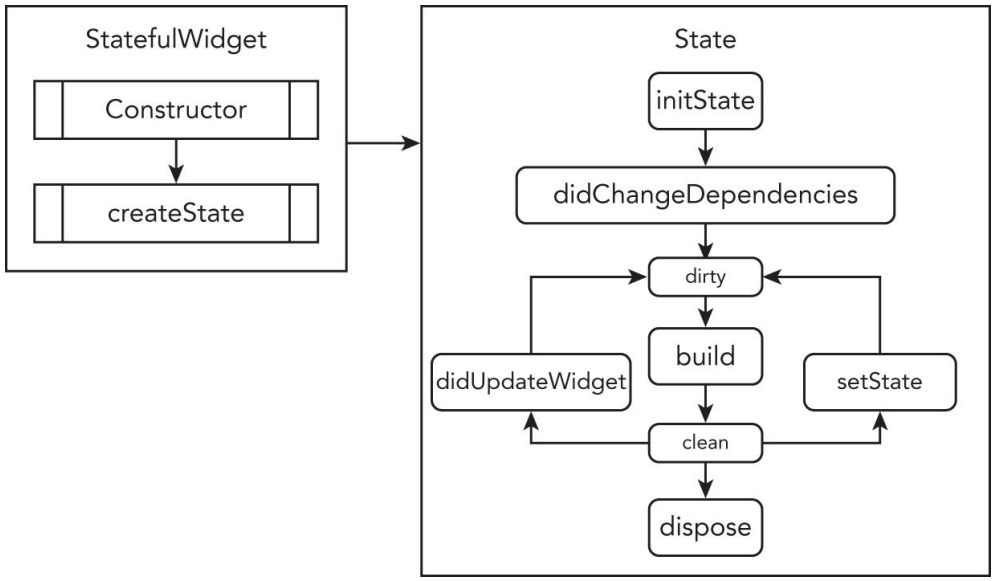
1. Виджет — основная единица построения интерфейса.
2. Композиция вместо наследования: собираем UI как конструктор.
3. Виджеты макета: `Padding`, `Center`, `SizedBox`.
4. Виджеты контента: `Text`, `Image`, `Icon`.
5. Виджеты тем: `ThemeData`, `Colors`.
6. Вложенность: иерархия «родитель — ребенок» (Child/Children).
7. Декларативный подход: мы описываем *что* хотим видеть, а не *как* это рисовать



Flutter

Stateless vs Stateful

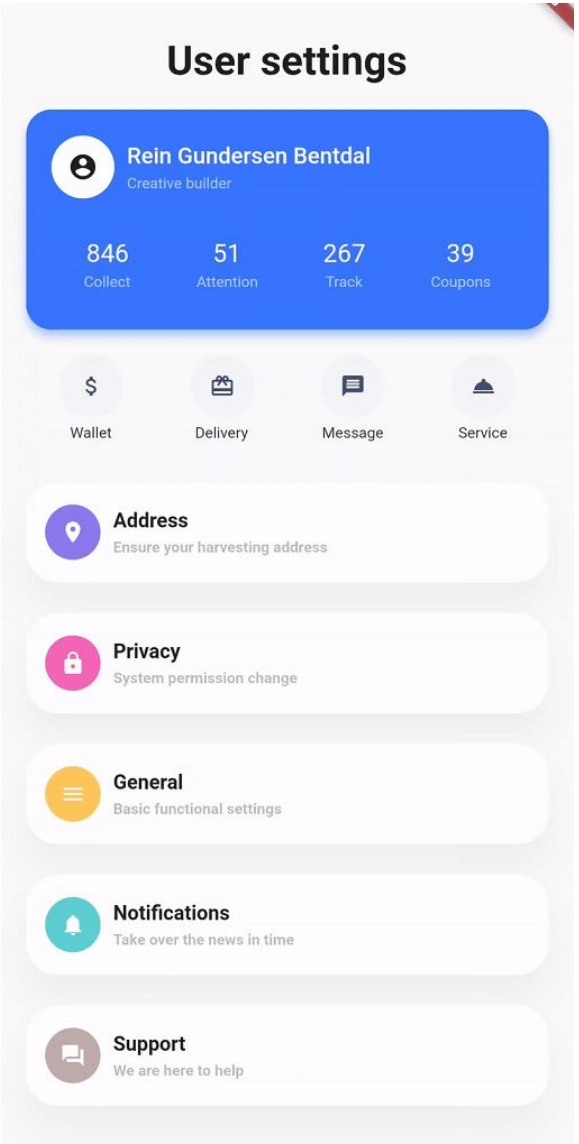
1. **Stateless:** Константные данные, не меняются после отрисовки.
2. Высокая производительность и малый вес.
3. **Stateful:** Имеет объект State, который живет дольше виджета.
4. Метод `setState()`: уведомляет фреймворк о необходимости обновления.
5. Жизненный цикл (Lifecycle): от `initState` до `dispose`.
6. Использование: формы ввода, таймеры, переключатели.
7. Разделение логики и представления внутри Stateful



Flutter

Основные визуальные виджеты

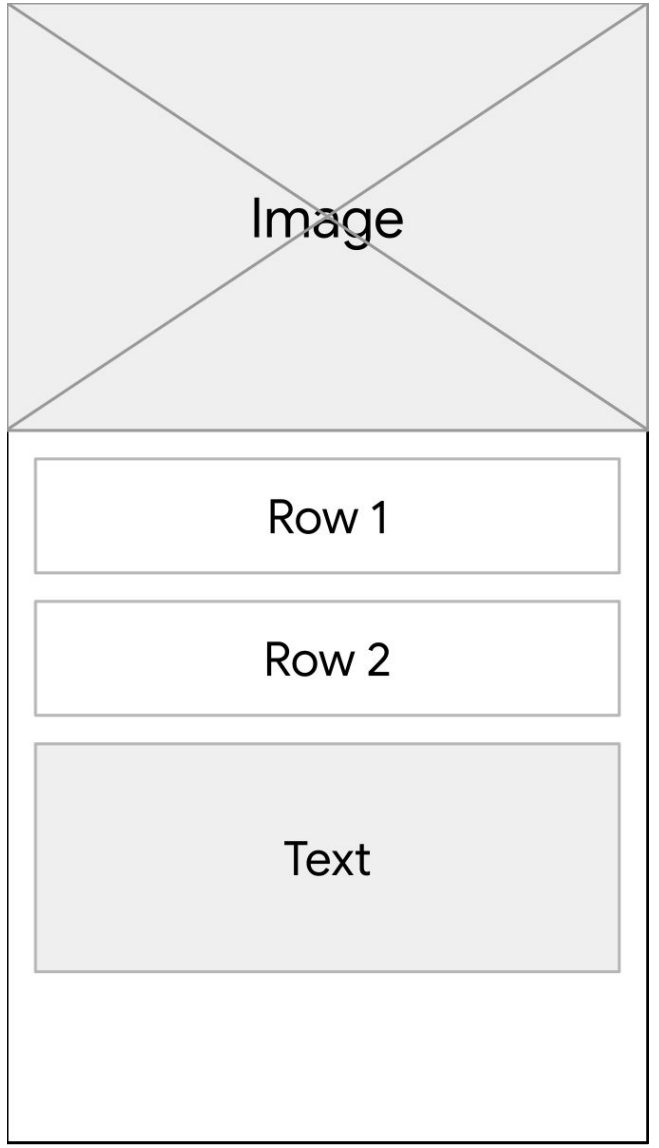
1. `Text`: стилизация, шрифты, перенос строк.
2. `Image`: загрузка из сети, ассетов или памяти.
3. `Icon`: огромный встроенный набор Material Icons.
4. `Container`: универсальная коробка с цветом, отступами и границами.
5. `Scaffold`: каркас экрана (шапка, подвал, боковое меню).
6. `Buttons`: `ElevatedButton`, `TextButton`, `OutlinedButton`.
7. `TextField`: ввод текста с полной валидацией



Flutter

Layout-виджеты

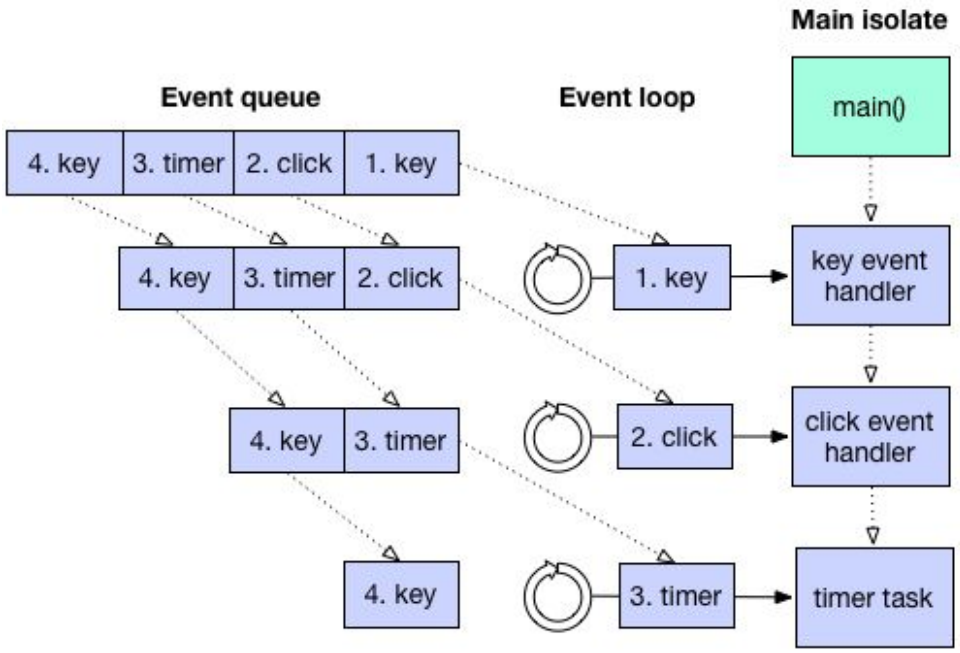
1. `Column`: вертикальный стек элементов.
2. `Row`: горизонтальная строка.
3. `Stack`: наложение слоев (например, текст поверх фото).
4. `Expanded`: заставляет элемент занять все свободное место.
5. `ListView`: автоматическое создание прокручиваемого списка.
6. `GridView`: сетка (например, для галереи фото).
7. `Padding`: управление внутренними отступами без создания сложных стилей.



Flutter

Event Loop и асинхронность

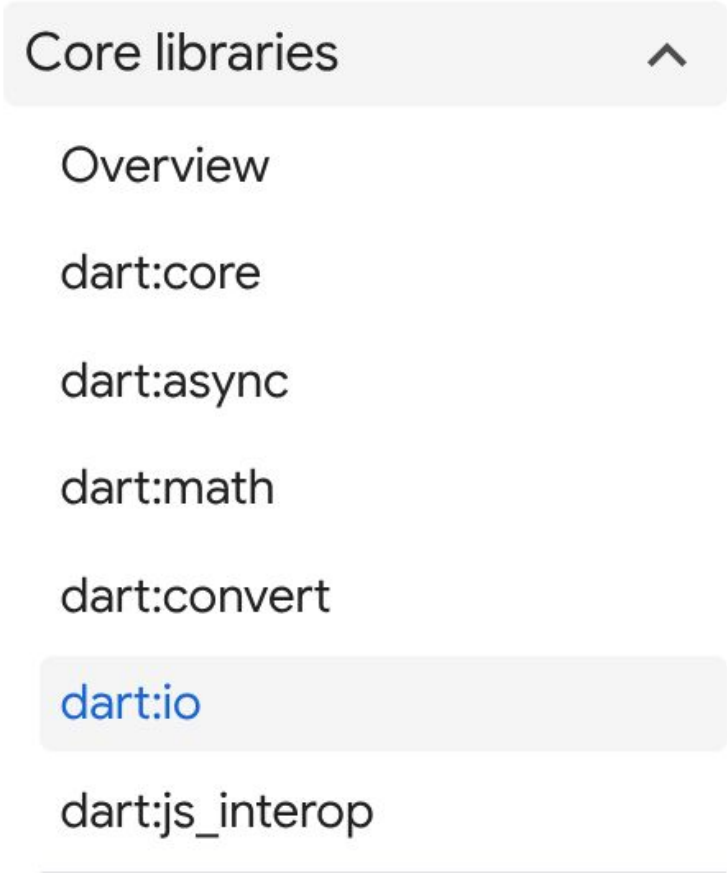
- **Однопоточность:** Dart работает в одном потоке (Isolate), что исключает конфликты памяти (race conditions).
- **Event Loop:** бесконечный цикл, который по очереди берет задачи из очередей и выполняет их.
- **Event Queue:** очередь события.
- **Microtask Queue:** внутренние критические задачи Dart
- **Асинхронность (Future):** способ поставить задачу в очередь, не замораживая интерфейс.
- **Isolates:** решение для тяжелых вычислений..
- **Плавность:** Event Loop позволяет обрабатывать ввод пользователя между кадрами отрисовки



Flutter

Стандартная библиотека

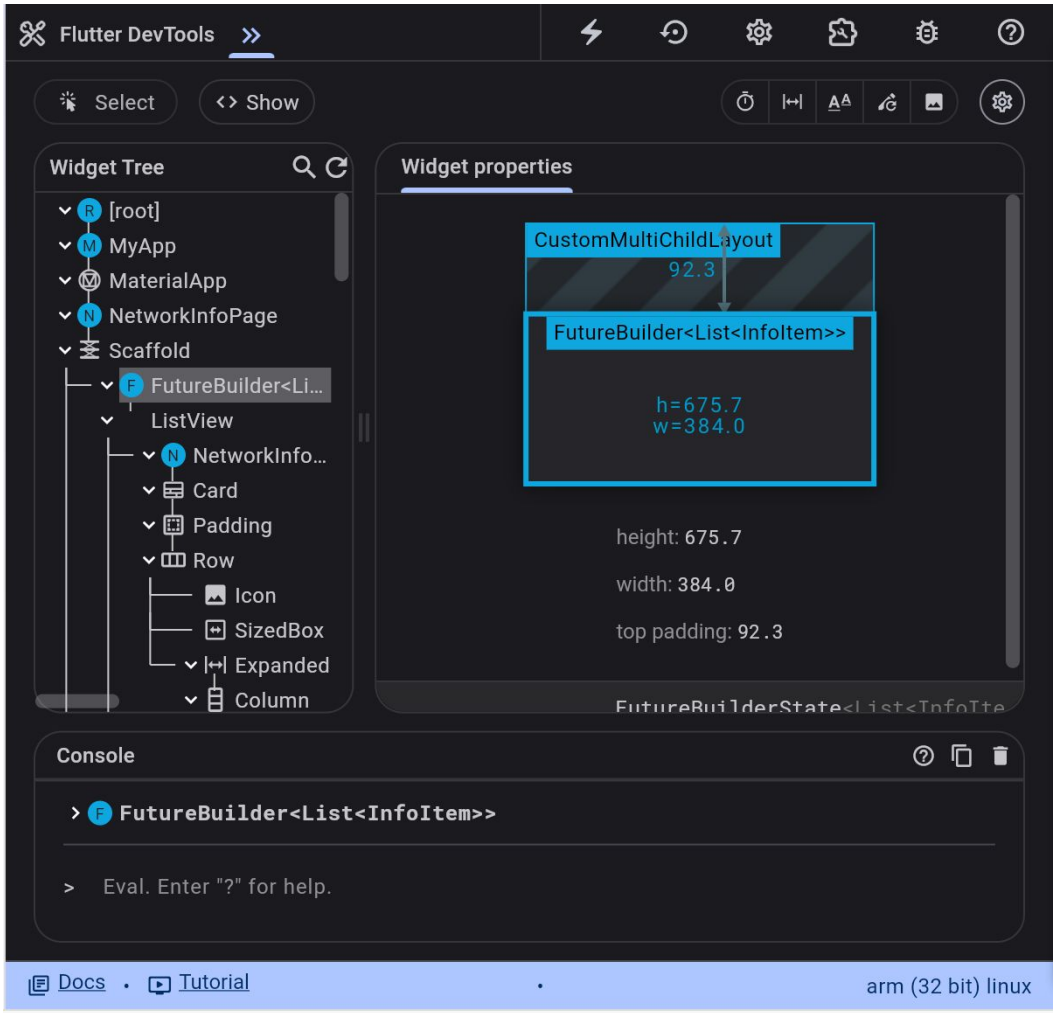
- **dart:io:** Файловая система, процессы, HTTP-клиент и сокеты.
- **dart:convert:** Встроенная поддержка JSON, UTF-8 и других кодировок.
- **dart:async:** Инструменты для асинхронного программирования: Future, Stream, Timer.
- **dart:math:** Математические константы, генерация случайных чисел и тригонометрия.
- **Internationalization (I10n):** Поддержка локализации, форматов дат и валют.
- **Canvas & Painting:** Низкоуровневый доступ к рисованию (CustomPainter) и анимациям.



Flutter

Dart Dev tools

1. **Hot Reload:** сохранение состояния при изменении кода.
2. **Hot Restart:** полная перезагрузка логики за 1-2 секунды.
3. **Widget Inspector:** возможность «ткнуть» в экран и найти виджет в коде.
4. **Network Profiler:** мониторинг всех запросов к серверу.
5. **Performance Overlay:** графики нагрузки на CPU/GPU в реальном времени.
6. Интеграция с VS Code, IntelliJ IDEA и Android Studio



Flutter

Пакеты и плагины

Пакеты vs Плагины

- **Dart Packages:** Чистый код на Dart. Расширяют логику приложения (например, *http*, *intl*).
- **Plugins:** Доступ к функциям ОС (камера, GPS, датчики) через нативные API (Android/iOS).

Источники:

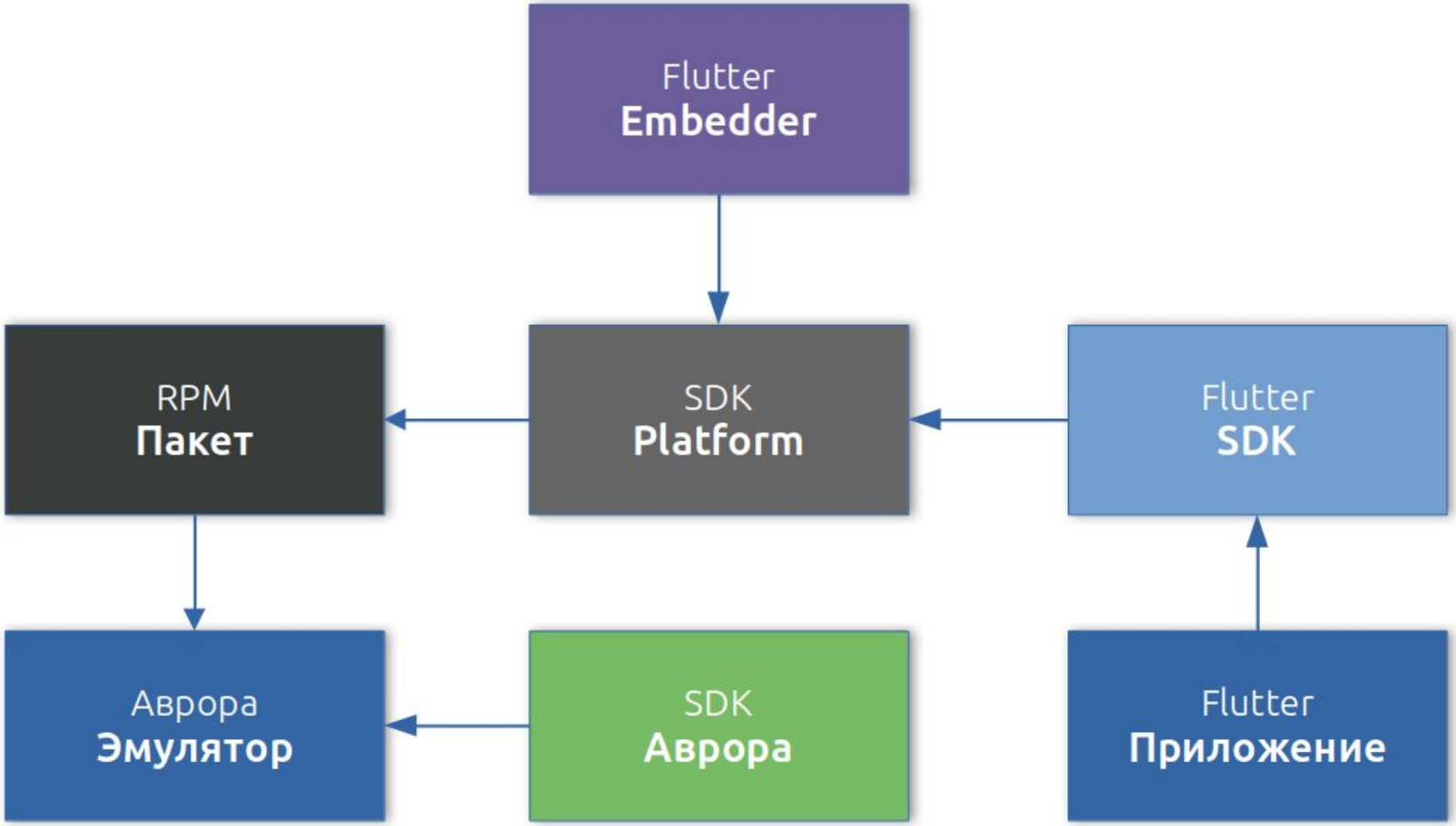
- pub.dev
- github, gitlab
- Приватные pub



Устройства Flutter для ОС Аврора

Flutter для ОС Аврора

Архитектура Flutter для ОС Аврора



Flutter для ОС Аврора

Адаптированные версии

3.27.3

3.29.2

3.32.7

3.35.7

Flutter для ОС Аврора

Что нужно для разработки



Flutter SDK



Aurora PSDK



Aurora SDK

Flutter для ОС Аврора

Пакеты и плагины

- Flutter SDK для ОС Аврора
- Пакеты с pub.dev
- Плагины от ОМП и Сообщества:
  - Исходники плагинов

Плагины:

- 30+ адаптированных плагинов
- 5 плагина для ОС Аврора

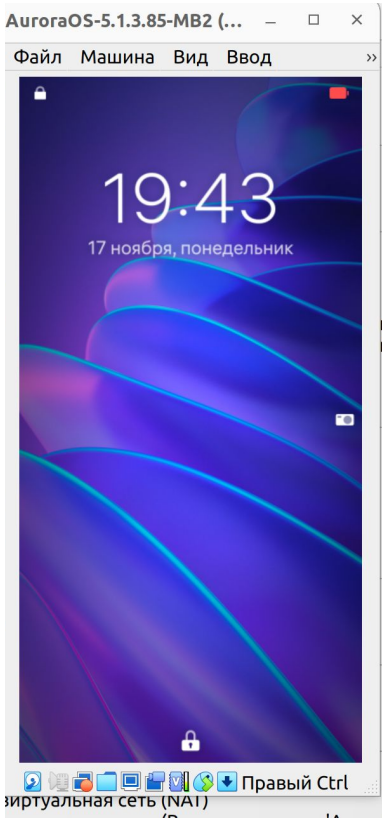


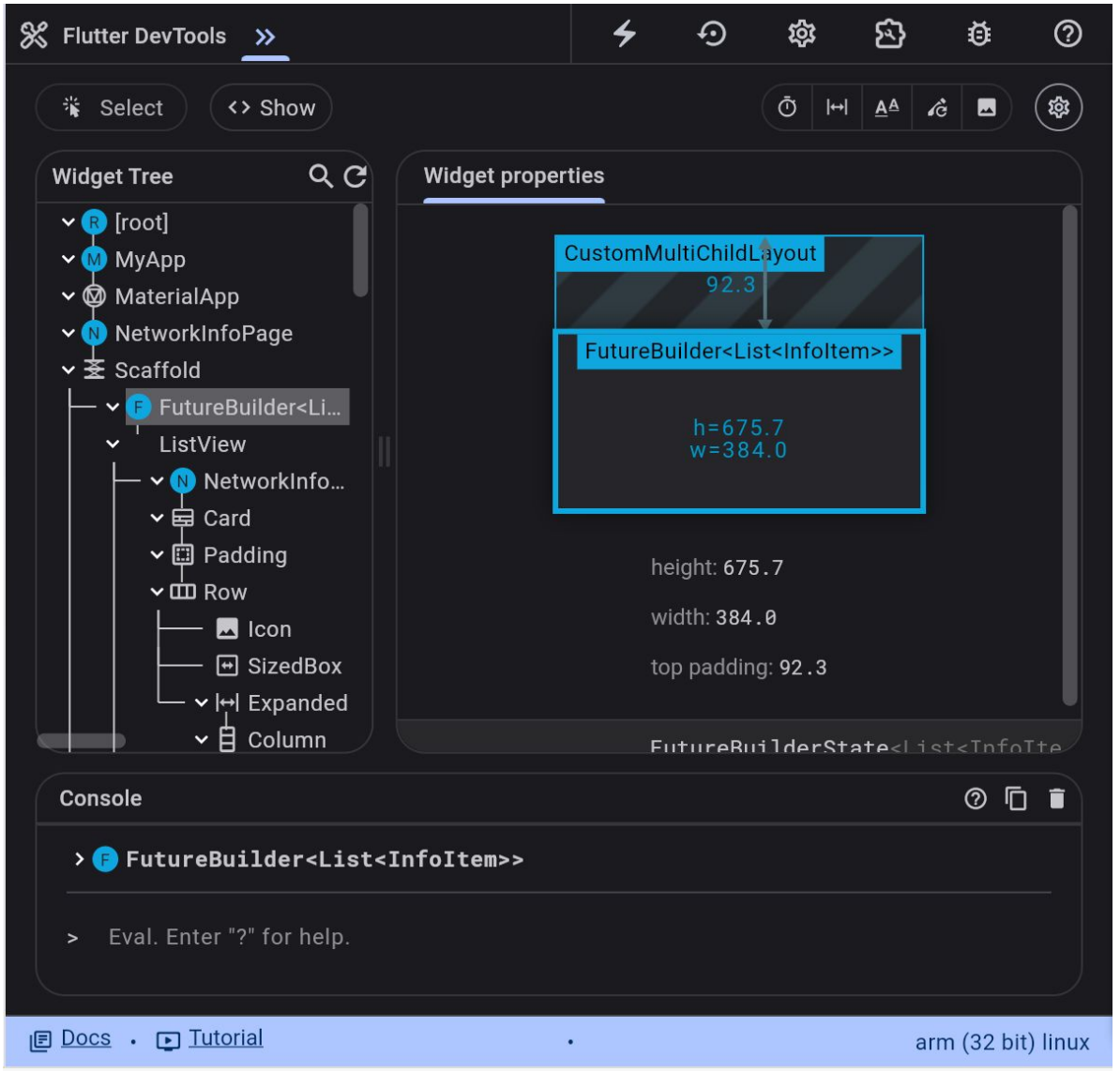
Исходники плагинов

Flutter для ОС Аврора

Инструментарий

- Dart Dev Tools
- Нативная отладка
- Эмулятор
- Инструменты для сборки





Flutter для ОС Аврора

Старт разработки

Создание приложения:

```
flutter-aurora create --platforms=aurora --template=app --org=ru.aurora app_demo
```

Получение зависимостей:

```
flutter-aurora pub get
```

Запуск приложения:

```
flutter-aurora run
```

Сборка RPM:

```
flutter-aurora build aurora --release --target-platform aurora-arm64
```

Особенности разработки под ОС Аврора

Особенности разработки под ОС Аврора

Desktop файл

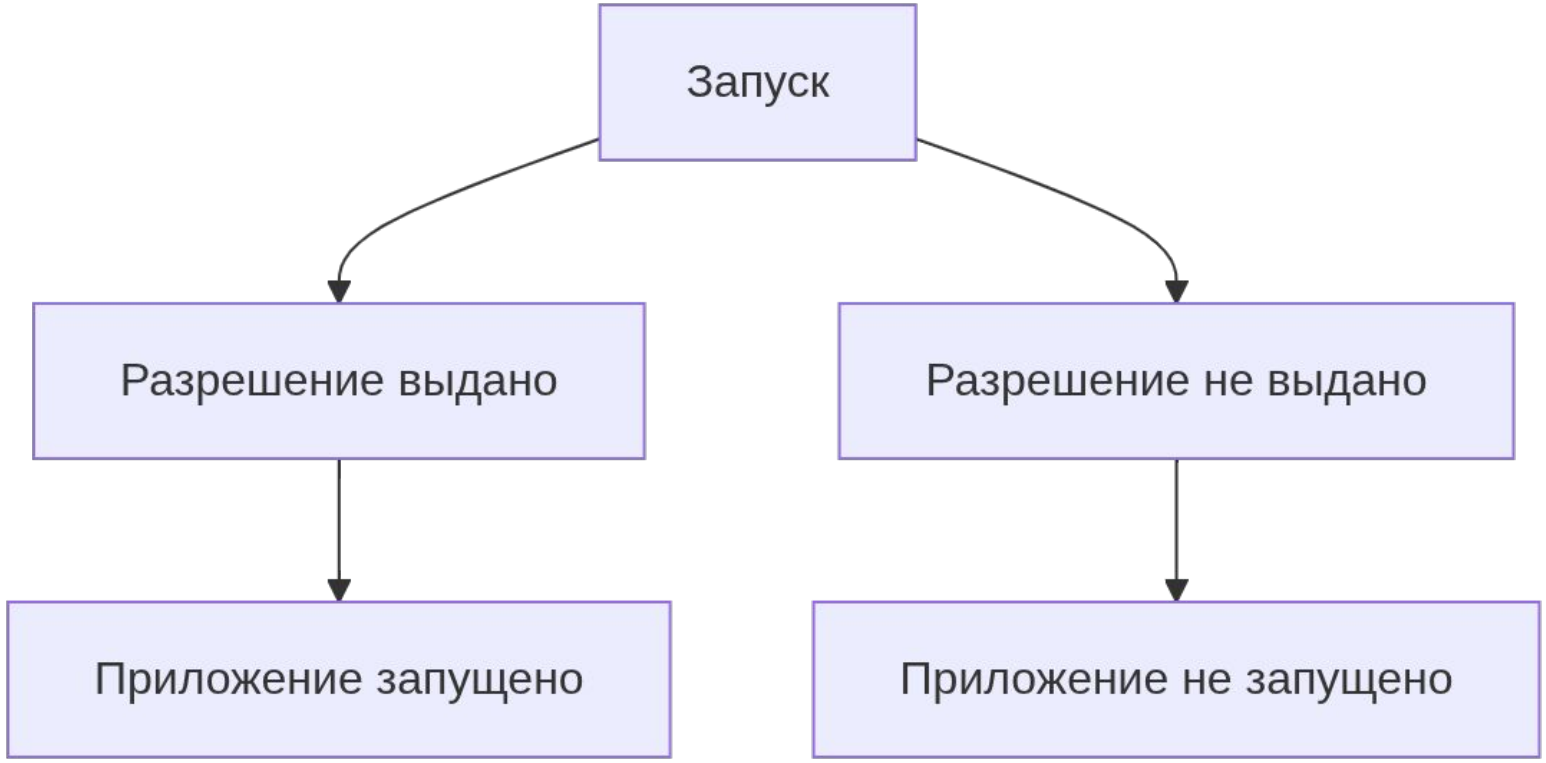
Документация

```
Type=Application
X-Nemo-Application-Type=silica-qt5
Icon={название_пакета}
Exec={команда_запуска}
Name={отображаемое_название_приложения_на_английском_языке}
Name[ru]={отображаемое_название_приложения_на_русском_языке}

[X-Application]
Permissions={список_разрешений}
OrganizationName={доменное_имя}
ApplicationName={название_приложения}
ExecDBus={команда_запуска_для_службы_D-Bus}
ExportDBusInterfaces={название_интерфейса_1};{название_интерфейса_2};...
```

Особенности разработки под ОС Аврора

Механизм работы разрешений



Особенности разработки под ОС Аврора

Валидация RPM пакета

Отключение валидации при разработке:

Настройки -> Средства разработчика -> Включить валидацию пакетов

Ручная проверка:

```
rpm-validator -p {профиль_безопасности}
  [параметры запуска] {путь_к_RPM-пакету}
```

Ключи для подписи:

- По умолчанию подпись regular ключом
- Возможность настройки своего ключа



Требования к RPM

Полезные ссылки

Скачать Flutter

Документация Flutter

Чат поддержки

Спасибо за внимание!